Google Photos Transfer / Organizer Project

Master Project Notes, Experiments, Failures, Findings & Next Steps

Prepared from the project conversation and the uploaded DOM diagnosis log
Date: 15 August 2026

1. Executive Summary

This document is the restart/reference document for the Google Photos Organizer / Transfer project. Its purpose is to preserve the complete reasoning path so a future fresh chat can understand what the project is trying to accomplish, what was tested, what worked, what failed, what Google Photos revealed through Playwright/CDP, and what remains unresolved.

The current project is NOT yet a finished transfer tool. The individual-media selection mechanism has been partially demonstrated, while reliable chronological traversal and automated album creation still need work.

2. Original Goal

The user has two Google accounts, Account A and Account B. Both are the user's accounts and both have sufficient Google Photos storage. The actual desired result is to copy photos and videos from Account A to Account B without using a temporary local download/upload workflow.

- Only Google Photos photos and videos matter; other Google account data is irrelevant.
- The collection is roughly 50 GB and spans about 2–3 years.
- Original quality is required.
- Photos and videos should be transferred.
- Duplicates are acceptable. If a duplicate occurs, it is not considered a problem.
- The user can manually accept/confirm actions on Account B.
- The user wants Account A organized into year-based albums such as 2025, primarily to make cross-verification easier.
- A shared-album mechanism was tested manually and works: sharing an album to the other account and accepting/adding it to the other account is viable.
- The automation focus became Account A organization/album creation rather than fully automating Account B acceptance.

3. Preferred Transfer / Organization Concept

The user initially considered one album per year, e.g. an album named 2025 containing all 2025 media. Later, because Google Photos selection can be limited and because a very large selection may be inconvenient, the project evolved toward batching selections into albums.

- Initial conceptual batch: one year = one album.
- Later test batch: 50 media items per album, then move to the next 50.
- Final intended batch concept: approximately 20,000 selected media at a time, with an album created before continuing.
- The final tool should preserve the chronological order from newest to oldest.
- Year boundaries should eventually be respected, so the project can create clear year-based organization or year-aware batches.

4. Technical Environment

- Windows desktop.
- Python.
- Existing virtual environment (venv) in the project.
- Playwright.
- Connection to an already logged-in Google Photos browser using Playwright CDP, typically via http://127.0.0.1:9222.
- The user manually logs into Google Photos. The automation does not need to automate Google authentication.
- No need for Account B automation at this stage; Account B acceptance can remain manual.
- The user asked that future scripts explain what they do in a concise overall summary so the project remains understandable.

5. Important Security / Safety Decision

The project intentionally uses the already logged-in browser session rather than trying to automate Google's login page. A previous login attempt through automation redirected to an About page / did not permit login. The chosen approach was to manually log in and then attach Playwright to the active browser session.

- This avoids trying to automate Google's authentication flow.
- No passwords or authentication secrets should be placed in source code.
- The scripts tested so far perform UI actions only in the logged-in Google Photos session.
- Destructive actions such as delete have deliberately not been used.
- Several diagnostic scripts explicitly reported that nothing was selected/deleted/downloaded/shared.

6. Key Google Photos UI Findings

6.1 Day-level selection exists

Google Photos exposes a checkbox for many day groups with labels such as 'Select all photos from Fri 26 Dec 2025'. This checkbox can select an entire day.

- The day checkbox uses role=checkbox.
- Its state is exposed through aria-checked='true' or aria-checked='false'.

- Clicking the exact checkbox through DOM/event interaction successfully selected a whole day.
- However, relying on the day checkbox became problematic because one-photo days and multi-photo days can have different DOM structures.

6.2 Individual media also expose selection state

Individual photos and videos also expose checkbox state through aria-checked. This is an important stable verification mechanism.

```
aria-checked="true"
aria-checked="false"
```

The uploaded diagnosis confirmed individual media and day-level selectors both appear as checkbox-like elements, but the semantic meaning is determined by the aria-label.

The diagnosis contained examples where day selectors were checked while their individual media checkboxes were still false. This is useful evidence that the day checkbox represents a higher-level selection state rather than being the same object as the media checkbox.

6.3 Individual media labels

Google Photos exposes individual media with aria-labels such as:

```
Photo – Portrait – 7 Aug 2026, 10:02:21
Video – Portrait – 7 Aug 2026, 18:02:02
```

The diagnosis also showed href values such as './photo/AF1Qip...'. These hrefs are much stronger candidates for identifying an individual media item than a visible timestamp/label.

6.4 Same timestamp does NOT mean duplicate

The user explicitly decided that two media items with identical visible labels/timestamps must still be treated as separate files. The automation must not deduplicate based on filename, timestamp, media type, or visible label.

```
Video – Portrait – 26 Jun 2026, 09:48:44
Video – Portrait – 26 Jun 2026, 09:48:44

These can be two separate media items and BOTH should count.
```

The only tracking problem is avoiding reprocessing the same DOM/media item after virtual scrolling. That is an automation traversal issue, not a duplicate-content policy.

6.5 Relative date labels

Google Photos does not always display full dates. Recent sections can use labels such as Today, Yesterday, Wednesday, etc. Older sections can use labels such as 'Sat 1 Aug', while a month heading can simply be 'July'. Therefore, the final traversal logic should not depend on exact full-year text appearing in every visible heading.

7. Timeline / DOM Findings

The main Google Photos timeline was discovered by inspecting scrollable elements. One large scrollable element had a very large scrollHeight, e.g. around 299,439 in one run, with a viewport/client height around 615. Another run reported around 292,937, demonstrating that these values are dynamic and must not be hard-coded.

- Do NOT assume a fixed 600-pixel scroll is universal.
- A 600-pixel test happened to move from late December 2025 to mid-December 2025 in one run, but this is not a universal relationship.
- The user specifically requested adaptive scrolling rather than system/display-specific pixel assumptions.
- A later adaptive scanner successfully moved backward through 2025 and reached January 2025.

8. Important DOM Concept: What is the DOM?

The DOM (Document Object Model) is the browser's structured representation of the webpage. Playwright interacts with DOM elements such as DIVs, links, buttons, checkboxes, and their attributes. In this project, the DOM is how the script discovers Google Photos media and selection controls without downloading the actual media files.

- HTML creates the page structure.
- The browser turns that structure into the DOM.
- Playwright queries the DOM and triggers actions on DOM elements.
- Google Photos is a highly dynamic application, so elements can be created, removed, virtualized, or moved as the timeline scrolls.

9. Diagnostic / Test Scripts and What They Did

Script / Experiment	Purpose / Result
inspect_photos.py	Connected to Google Photos and inspected page title, visible text, buttons, links and aria-labels. It showed the Google Photos UI but initially did not find visible '2025' text.
scroll_test.py	Tested scrolling the timeline until a 2025 marker became visible. It proved that scrolling could expose 2025.
inspect_2025.py	Inspected visible aria-label elements around 2025. It showed day-level selectors and individual Photo/Video labels. The visible DOM contained only a limited window of media, not all 2025 media.
timeline scan / 150-scroll experiment	Scanned the timeline repeatedly and discovered only a limited set of unique visible media because the script was based on what remained detectable in the DOM.
select_one_day.py	Tested selecting a single day-level checkbox. Normal Playwright click/hover was sometimes intercepted by photo overlays. DOM/event-based clicking eventually succeeded.
select_6_days.py	Tried selecting multiple day checkboxes. Only the first loaded day was available; other target days

	were not in the DOM at that moment. This demonstrated that virtualized loading makes global selectors unreliable.
selection_one_day_diagnosis.py	Compared day selectors before and after selection. It successfully showed that the target day's aria-checked changed from false to true and back to false during the read-only test.
find_scroll.py	Inspected scrollable elements and identified a huge main timeline scroll container. The dimensions changed between runs, reinforcing the need for adaptive logic.
test_timeline_scroll.py	Demonstrated that moving the main timeline by a controlled amount changed the loaded date window. It was later adapted to use viewport-relative movement.
timeline_scroll_adaptive.py	Read-only scanner that adaptively scrolled backward and discovered 73 unique dates in 2025, eventually reaching January 2025.
position_test.py	Moved to the timeline top and observed that the top represented current/recent dates. It helped establish the newest-to-oldest direction.
2025 position search experiment	A position-based search initially stopped at an artificial search limit before reaching 2025. Increasing the search allowed 2025 to appear around a timeline position near 142,726 in one run.
selected_files_count.py	Selected currently loaded August selectors. It exposed the problem that not all August days were loaded and that relative labels such as Today/Yesterday/Wednesday were mixed with date labels.
one_day_understanding.py / diagnosis	Deeply inspected ancestors, siblings, day containers, media counts, checkboxes and hrefs for 5/6/7 August. This produced the most useful DOM evidence so far.
50_batch.py	First 50-item batch experiment. It successfully selected 50 items but failed to create an album because the assumed Create button was not visible. It also exposed duplicate-looking timestamps and a traversal problem.
50_checkpoint_album_test.py / later batch attempts	Attempted checkpoint-based 50-item batches and album creation using the manually described workflow. The traversal still behaved incorrectly, so this is not yet reliable.

10. Day-Level Selection Experiments

10.1 First failure: normal click timed out

```
Locator.click: Timeout 30000ms exceeded
element resolved to <div role="checkbox" aria-label="Select all photos from Fri, 26 Dec
2025">...
element is not visible / pointer events intercepted
```

The element could exist in the DOM but be visually covered by a photo tile or overlay. Normal Playwright pointer actions were therefore unreliable.

10.2 DOM/event click succeeded

A later version checked aria-checked, dispatched the click on the exact Google Photos checkbox, then checked aria-checked again.

```
Checkbox before click:
aria-checked = false

Triggering click on the exact Google Photos checkbox...

Checkbox after click:
aria-checked = true

✓ 26 Dec 2025 is selected.
```

This proved that event-based selection can work even when normal pointer interaction is blocked by overlays.

10.3 Multiple day selectors failed because of virtualization

When trying six dates, only the currently loaded date was found. The other five were NOT FOUND. A later diagnosis showed only three day selectors were loaded at one point: 26, 27 and 28 Dec. This established that a date can exist in the timeline but not currently exist in the DOM.

11. The 5/6/7 August Problem

This was a key discovery. The user noticed that some days have a day-level selection tick, while a day containing only one photo may show the selection control on the individual photo instead of as a separate day-level checkbox.

The diagnosis confirmed that the DOM structure differs. For 7 Aug, the date heading had a nearby day-level checkbox and a day container containing four media items. For 5 Aug, the diagnosis showed a container with one media item whose checkbox had the media aria-label itself. This is why a simplistic 'find date heading -> click parent checkbox' strategy failed.

The diagnosis also showed a larger ancestor that contained many unrelated dates, so using an arbitrary ancestor level was unsafe.

This is the reason the project eventually moved away from day-level selection and toward individual media selection.

Source evidence from the uploaded diagnosis: 7 Aug had a day selector plus four media items; 5 Aug had a one-media container; broader ancestors contained many dates and media. □filecite□turn3file9□ □filecite□turn3file10□

12. The Individual-Media Approach

The user proposed a simpler approach: ignore day-level selection completely and select every individual photo/video.

- This avoids inconsistent day-checkbox structures.
- One-photo days are naturally handled because the photo itself is selected.

- Multi-photo days are handled by selecting each media item.
- Photo and video labels can both be handled.
- aria-checked can be used to verify selection.
- Duplicate-looking files remain separate.
- The final batching logic can count exact selected media instead of relying on day counts.

13. First Successful 50-Item Selection Test

A 50-item batch script successfully reached 50 selected media. The output included:

```
Selecting 48/50: Photo – Portrait – 26 Jun 2026, 20:16:26
Selecting 49/50: Video – Portrait – 26 Jun 2026, 09:48:44
Selecting 50/50: Video – Portrait – 26 Jun 2026, 09:48:44
Progress: 50/50
```

This demonstrated that the individual selection mechanism can reach a target count. However, it also showed that the same visible label can occur twice, which the user correctly stated should NOT be deduplicated.

The same test then failed at album creation because the script assumed a particular Create control was visible. The script correctly stopped without clearing the 50 selected items.

14. Correct Album-Creation Workflow Confirmed Manually

The user manually clarified the actual Google Photos workflow. This should be treated as the authoritative workflow for the automation:

1. 50 photos/videos are selected.
2. Click the + button.
3. Choose Album.
4. Choose Create album.
5. Enter the album name.
6. Click the checkbox in the upper-left corner.
7. Click Back at the same position.
8. Google Photos creates the album.
9. Google Photos automatically deselects the selected files.

The automation should verify the automatic deselection before beginning the next batch. There is no need to manually clear the selection if Google Photos has already cleared it.

15. Checkpoint / Resume Idea

The user asked whether the script can remember the last checked picture and continue from the same timeline position after album creation.

The proposed design is to save a checkpoint containing at least:

```
{
  "album_number": 1,
  "last_media_href": "...",
```

```
  "last_media_label": "...",
  "timeline_position": 123456,
  "selected_count": 50
}
```

- The checkpoint is for resuming automation, not for deduplicating real user media.
- The last media should preferably be identified by its Google Photos href when available.
- The timeline position is a second piece of state.
- After album creation, Google Photos should auto-deselect.
- The script should verify deselection.
- The script should resume at the same timeline position and continue toward older media.

16. Major Failure: Chronological Traversal

The latest batch attempt exposed the biggest unresolved issue. The script could select 50 items, but it did not reliably move strictly from today toward the past. It could appear to jump from around 7 Aug to another part of the timeline, and in one run only 45 eligible files were available before the script incorrectly concluded it could not reach 50.

The user explicitly clarified the desired direction: start from NOW/today and move backward into the past. If the current viewport has only 45 files, the script must continue scrolling backward and find the remaining 5 instead of stopping.

A later attempted fix added a date guard such as media_date <= previous_date, but the user reported that the approach still failed. Therefore, chronological traversal is NOT considered solved.

17. Why Chronological Traversal Is Hard

- Google Photos virtualizes the timeline: not every media item is present in the DOM at once.
- Scrolling changes which days/media are loaded.
- Visible DOM order is not sufficient as a global chronological order.
- Large ancestor containers can contain media from multiple days.
- The timeline scroll position changes dynamically and differs between runs.
- A fixed pixel scroll such as 600 is not a reliable semantic step.
- The page can temporarily expose overlapping old/new media around the viewport.
- Relative labels such as Today/Yesterday/Wednesday complicate heading-based parsing.
- The script must not treat identical visible labels as duplicates.
- A date-based guard alone cannot prove that every media item between two scroll positions was visited.

18. What Worked

- Manual Google Photos shared-album transfer concept works.
- Manual login + Playwright CDP connection works.
- Google Photos can be inspected through Playwright.
- The main timeline scroll container can be discovered dynamically.

- Adaptive scrolling can move through large portions of the timeline.
- A read-only scanner successfully reached January 2025.
- Day-level checkbox can be selected with DOM/event click in some cases.
- Individual media can be selected and verified with aria-checked.
- A 50-item selection test reached exactly 50.
- The script can safely stop without clearing selection when album creation is uncertain.
- Google Photos exposes hrefs for individual media, which is promising for identity/checkpointing.
- The actual album workflow has now been manually established.

19. What Failed / Should NOT Be Reused as Final Logic

Failed approach	Why it failed / final decision
Automating Google login	Did not reliably allow login and redirected to an About page. Use an already logged-in browser session instead.
Fixed date text search for 2025	Can require a lot of scrolling and is unreliable because date/year labels are virtualized.
Assuming all target day selectors are simultaneously in DOM	False; many dates are not loaded until the timeline is scrolled.
Normal Playwright click/hover on day checkbox	Can time out because photo overlays intercept pointer events.
Arbitrary ancestor-level DOM selection	Unsafe because a larger ancestor can contain many unrelated dates/media.
Day-level checkbox as the universal method	Fails for one-photo days and inconsistent DOM structures.
Deduplicating by label/timestamp	Rejected by user; two separate media can share identical visible labels.
Fixed 600-pixel scrolling	Works in a test but is not adaptive and depends on current viewport/content.
Assuming visible media equals the next chronological batch	False because of virtualization and overlapping loaded elements.
Stopping because current visible/new media count is only 45	Wrong; the script must continue scrolling until the target batch is filled.
Assuming a generic 'Create and add photos' button is the album button	Failed because it was not visible in the selection state.

20. Current Authoritative Requirements

- Use Account A's Google Photos.
- Only photos and videos are relevant.
- Original quality is required for the eventual transfer workflow.
- Account B acceptance can remain manual.
- The user wants year-oriented organization for cross-verification.
- Selection should be individual media rather than relying on day checkboxes.
- Every separate media item counts, even if another item has the same timestamp/label.
- Use aria-checked to verify whether an individual item is selected.
- Start from today/current newest media and move backward toward the past.
- Never randomly jump to a later/newer date after the traversal has moved backward.
- If a viewport has fewer than the target batch size, keep scrolling until enough media are found.

- For the test, use 50 items per album.
- For the eventual system, target batches around 20,000 or another verified safe limit.
- After selecting a batch, create an album using the manually confirmed UI sequence.
- Google Photos automatically deselects after album creation; verify this before continuing.
- Resume from the same timeline position after each album.
- Save a checkpoint so a crash/restart does not require starting over.
- Do not delete anything.
- Do not download media as part of this approach.
- Do not share automatically unless explicitly added later.

21. Current Best Architecture

The strongest architecture suggested by the experiments is a stateful, newest-to-oldest media iterator with checkpointing. The important part is that the iterator must be based on the actual timeline's progression, not simply the set of visible DOM nodes.

```
START
  ↓
Attach to logged-in Google Photos
  ↓
Find main timeline dynamically
  ↓
Move to newest/top
  ↓
Observe current chronological boundary
  ↓
Collect/select individual media currently at or before boundary
  ↓
Move timeline backward adaptively
  ↓
Wait for new older content to load
  ↓
Repeat until batch reaches 50 (test) / 20,000 (future)
  ↓
Save checkpoint: last media + position + batch info
  ↓
Create album: + → Album → Create album → name → upper-left checkbox → Back
  ↓
Verify Google Photos auto-deselected
  ↓
Resume at same timeline position
  ↓
Continue backward
  ↓
Year boundary / end of requested historical range
  ↓
FINISH
```

22. What the Next Fresh Chat Should NOT Assume

- Do not assume the previous scripts are correct; several were exploratory tests.
- Do not restart by proposing day-level selection without considering the 5/6/7 August findings.
- Do not deduplicate identical labels/timestamps.
- Do not use fixed scroll pixels as a final strategy.

- Do not assume all dates are in the DOM simultaneously.
- Do not assume a visible count of 45 means only 45 remain in the batch.
- Do not clear a selection if album creation failed.
- Do not assume the generic Create button is the album workflow.
- Do not jump directly to a 20,000-item production run before the 50 → album → next 50 test is reliable.

23. Most Important Unresolved Problems

- Reliable chronological traversal from today toward the past.
- A robust way to determine which currently loaded media are truly the next media in chronological sequence.
- Reliable handling of Google Photos virtualized DOM without skipping media between scroll positions.
- Reliable individual-media identity using href/DOM relationship without treating same-label media as duplicates.
- Exact automated album UI implementation matching the manually confirmed + → Album → Create album → name → upper-left checkbox → Back workflow.
- Reliable verification that the album contains exactly the intended batch before proceeding.
- Checkpoint recovery after browser crash or script interruption.
- Year boundary handling for 2026 → 2025 → 2024 etc.
- Eventually testing at a larger batch size and verifying the practical 20,000-item limit before using it.

24. Suggested Next Experiment

Do NOT immediately build the full transfer tool. The next experiment should isolate the hardest problem: chronological traversal.

10. Start at timeline top/current media.
11. Read a small window of media and their dates/hrefs.
12. Move backward a small adaptive amount.
13. Record before/after media date ranges and hrefs.
14. Verify that the new window contains older media and identify the overlap.
15. Repeat for several steps without selecting anything.
16. Use the results to build a reliable 'cursor' concept representing the last processed media.
17. Only after traversal is proven, add selection.
18. Only after selection is proven, add album creation.

This separation will make debugging much easier: traversal, selection, and album creation can be tested independently instead of all failing at once.

25. Useful Evidence From diagnosis.txt

The uploaded diagnosis log is especially valuable because it provides concrete DOM evidence rather than assumptions. For example, it records a 7 Aug container with four media items and a day-level checkbox, while 5 Aug has a one-media container whose checkbox is the media itself. It

also shows hrefs for individual media and larger ancestors containing many dates. □filecite□turn3file9□ □filecite□turn3file10□

The diagnosis also records many individual Photo/Video checkbox entries and their aria-checked states, including examples of selected day selectors and unselected individual media. □filecite□turn3file0□turn3file1□

26. Restart Prompt for a Future Chat

Copy/paste the following at the start of a fresh chat together with this document:

```
I am continuing my Google Photos Organizer project. Read the attached master project
document first. Do not assume previous experimental scripts are correct. The goal is to
organize/transfer Google Photos photos and videos from Account A, newest to oldest,
using individual media selection, with batching and album creation. Account B acceptance
can remain manual. Duplicates are acceptable and identical visible timestamps must not
be deduplicated. The current biggest unresolved problem is reliable chronological
traversal through Google's virtualized timeline. The confirmed album workflow is: + →
Album → Create album → enter name → upper-left checkbox → Back; Google Photos then
automatically deselects. Use checkpoints and never delete/download/share. First explain
what is already proven, what remains unresolved, and propose the smallest next
diagnostic test.
```

27. Final Project Status

Area	Status	Notes
Google Photos browser connection	WORKING	Logged-in session via Playwright/CDP works.
Timeline discovery	WORKING	Large scrollable timeline can be detected dynamically.
Adaptive scrolling	PARTIAL	Works in experiments, but not yet reliable enough for exact media traversal.
2025 discovery	WORKING AS SCAN	A read-only adaptive scan reached January 2025.
Day-level selection	PARTIAL / REJECTED AS MAIN METHOD	Works sometimes, but DOM differs and virtualization makes it unreliable.
Individual media selection	PARTIALLY WORKING	A 50-item test reached 50.
aria-checked verification	WORKING	Confirmed useful for selection state.
Duplicate handling	DEFINED	Do not deduplicate same-label/timestamp media.
Album workflow	MANUALLY CONFIRMED	Exact sequence documented, automation still needs DOM implementation.
Checkpoint concept	DESIGNED	Not yet proven end-to-end.
50 → album → next 50	NOT YET RELIABLE	Traversal/album automation still unresolved.
20,000 production batches	NOT STARTED	Only after smaller test is reliable.
Account B acceptance	MANUAL	Explicitly acceptable to user.

28. Bottom Line

The project has progressed from a general transfer idea to a much more concrete UI-automation design. The strongest facts established are: Google Photos can be controlled through the logged-in browser session; the timeline is virtualized; individual media expose Photo/Video aria-labels and hrefs; selection state is observable through aria-checked; day-level selection is inconsistent for one-photo days; individual selection can reach a target count; and the real album workflow is now known.

The main thing still preventing a reliable final tool is not basic clicking. It is building a correct chronological cursor through Google Photos' virtualized timeline so that no media are skipped, repeated, or selected out of order. That should be solved in isolation before scaling the system.

End of Master Project Notes